WEEK 4 · LESSON 23 OF 24 · HANDOUT

Saving, Sharing and Monitoring Models

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=URdnFlZnlaE — open the lesson online to watch inside the course.

Your goal in this lesson: Save a trained pipeline with joblib, load it to make predictions, document it with a model card, and check new data for drift.

A model is only useful when it is used

A notebook model helps nobody. To be useful it must be **saved**, **loaded** by an app (a website, a USSD service, a mobile app) and **watched** over time. This step is called **deployment**, and the wider practice is **MLOps**.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

Train the final pipeline on all development data

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

pipe = make_pipeline(StandardScaler(), LogisticRegression(C=1.0))
pipe.fit(X_train, y_train)
test_auc = roc_auc_score(y_test, pipe.predict_proba(X_test)[:, 1])
print("Test AUC:", round(test_auc, 3))
```

Save the **whole pipeline**, not just the model — the scaler is part of the model. Saving only the model is a classic bug: the app then feeds it unscaled numbers.

Save and load with joblib

```
import joblib

joblib.dump(pipe, "loan_model_v1.joblib")

loaded = joblib.load("loan_model_v1.joblib")
same = (loaded.predict_proba(X_test) == pipe.predict_proba(X_test)).all()
print("Loaded model gives identical predictions:", same)
```

In Colab, download the file from the Files panel or save it to Google Drive.

Rules for model files:

- Load files only from sources you **trust** — a joblib/pickle file can run code when opened.
- Record the **scikit-learn version**: a model saved with one version may not load in a very different one.
- Use clear names with versions: `loan_model_v1.joblib`, `v2`…

A prediction function for the app

```
import pandas as pd

FEATURES = ["income_rwf", "loan_rwf", "months_customer", "late_payments", "salaried"]
THRESHOLD = 0.25

def decide(application: dict) -> dict:
    row = pd.DataFrame([application])[FEATURES]
    risk = float(loaded.predict_proba(row)[0, 1])
    return {"risk": round(risk, 3),
            "decision": "refer to officer" if risk >= THRESHOLD else "approve"}

print(decide({"income_rwf": 180_000, "loan_rwf": 90_000, "months_customer": 24,
              "late_payments": 0, "salaried": 1}))
print(decide({"income_rwf": 120_000, "loan_rwf": 250_000, "months_customer": 2,
              "late_payments": 3, "salaried": 0}))
```

Notice the decision for risky cases is **"refer to officer"**, not "refuse": a person reviews them. A web framework such as **FastAPI** (which powers the WAMUGII website) can wrap a function like this as an API.

Write a model card

A **model card** is a short document that travels with the model:

```
import json
import sklearn
from datetime import date

card = {
    "name": "Micro-loan default risk",
    "version": "v1",
    "date": str(date.today()),
    "sklearn_version": sklearn.__version__,
    "intended_use": "Rank mobile micro-loan applications by default risk; risky cases go to a
loan officer.",
    "not_for": "Automatic refusal without human review; loans above 300,000 RWF.",
    "training_data": "2,000 simulated past borrowers (practice data).",
    "features": FEATURES,
    "threshold": THRESHOLD,
    "test_auc": round(test_auc, 3),
    "fairness_check": "Refusal rates audited by region (see Lesson 22).",
    "retrain": "Every 3 months, or when drift is detected.",
}
with open("loan_model_v1_card.json", "w") as f:
    json.dump(card, f, indent=2)
print(json.dumps(card, indent=2)[:400])
```

Monitor for drift

The world changes: inflation raises incomes and loan sizes, a new competitor changes who applies. When new data looks different from the training data — **data drift** — the model's accuracy can quietly fall.

```
new_month = X_test.copy()
new_month["loan_rwf"] = (new_month["loan_rwf"] * 1.4).round(-3)    # loans grew 40%

comparison = pd.DataFrame({"training mean": X_train.mean(), "this month": new_month.mean()})
comparison["change %"] = ((comparison["this month"] / comparison["training mean"] - 1) *
100).round(1)
print(comparison.round(0))

before = (loaded.predict_proba(X_test)[:, 1] >= THRESHOLD).mean()
after = (loaded.predict_proba(new_month)[:, 1] >= THRESHOLD).mean()
print("Share referred: test data", round(before, 3), "-> this month", round(after, 3))
```

Loan sizes jumped about 40%, and the share of applicants referred to an officer rises from about a third to about half. That is an alarm: investigate, collect new labelled outcomes, and **retrain**.

What to monitor every month:

1. Input features — averages, ranges, missing values.
2. Predictions — the share approved or referred.
3. Real outcomes once known — recall, precision, AUC.
4. Fairness by group.

Key terms

- **Deployment** — putting a model into real use.
- **joblib** — a library to save and load Python objects such as pipelines.
- **Model card** — a short document describing a model's use, data, performance and limits.
- **Data drift** — new data that differs from the training data.

- **MLOps** — the practice of deploying, monitoring and maintaining models.

Remember this: Save the whole pipeline with joblib, wrap it in a simple prediction function with human review, document it in a model card and watch for drift so you know when to retrain.

Name: _____ Date: _____

Activity

Task 1

Save the homes pipeline from Lesson 21, load it, and predict the rent of three new homes.

Task 2

Add `"limitations"` and `"contact"` fields to the model card.

Task 3

Simulate a month where incomes rise 20% and loan sizes stay the same. Does the referral rate go up or down? Why?

Task 4

Write a one-paragraph monitoring plan for the SACCO: what you check, how often, and what triggers retraining.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why save the whole pipeline instead of only the model?

   A. The preparation steps (like the scaler) are part of the model and must be applied to new data
   B. Models cannot be saved
   C. Files are smaller
   D. It is faster to train

2. Which library is commonly used to save scikit-learn pipelines?

   A. matplotlib
   B. seaborn
   C. joblib
   D. requests

3. Why only load model files from trusted sources?

   A. They expire
   B. They are large
   C. Loading a pickle/joblib file can run code
   D. They contain passwords

4. What is a model card?

   A. A chart
   B. A credit card for AI
   C. The model file
   D. A short document describing a model's purpose, data, performance and limits

5. Loan sizes rise 40% after training. What is this called?

   A. Regularisation
   B. Overfitting
   C. Data drift
   D. Clustering

6. Why does the app say 'refer to officer' instead of 'refuse'?

   A. Because the model cannot refuse
   B. To keep a human in the loop for risky decisions
   C. It is a bug
   D. To make it slower

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.